LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Hafiz Perdana NIM. 2410817210027

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Hafiz Perdana
NIM : 2410817210027

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 1
B. Output Program 11
C. Pembahasan 12
SOAL 2 16
A. Pembahasan 16
ADDITIONAL 18
1. Fitur Unggulan Kotlin 18
2. Test Case: Penggunaan Fitur Null Safety 20
DAFTAR PUSTAKA 22
TAUTAN GIT 23

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 XML..... 11
Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose 12

DAFTAR TABEL

Tabel 1. Source Code MainActivity XML Soal 1 1
Tabel 2. Source Code activity_main XML Soal 1..... 3
Tabel 3. Source Code MainActivity Compose Soal 1 6

SOAL 1

Soal Praktikum:

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.

A. Source Code

MainActivity.kt - XML

Tabel 1. Source Code MainActivity XML Soal 1

```
package com.example.modul2xml

import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.widget.ArrayAdapter
import android.widget.AutoCompleteTextView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.materialswitch.MaterialSwitch
import com.google.android.material.textfield.TextInputEditText
import java.text.NumberFormat
import kotlin.math.ceil

class MainActivity : AppCompatActivity() {
    private lateinit var etBillAmount: TextInputEditText
    private lateinit var spinnerTip: AutoCompleteTextView
    private lateinit var switchRoundUp: MaterialSwitch
    private lateinit var tvTipAmount: TextView

```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        etBillAmount = findViewById(R.id.etBillAmount)
        spinnerTip = findViewById(R.id.spinnerTipPercentage)
        switchRoundUp = findViewById(R.id.switchRoundUp)
        tvTipAmount = findViewById(R.id.tvTipAmount)

        val options = arrayOf("15%", "18%", "20%")
        val adapter = ArrayAdapter(this, android.R.layout.simple_dropdown_item_1line, options)
        spinnerTip.setAdapter(adapter)

        spinnerTip.setText("15%", false)

        etBillAmount.addTextChangedListener(object : TextWatcher {
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}

            override fun afterTextChanged(s: Editable?) {
                calculateTip()
            }
        })

        etBillAmount.setOnEditorActionListener { v, actionId, event ->
            if (actionId == android.view.inputmethod.EditorInfo.IME_ACTION_NEXT) {
                val imm = getSystemService(android.content.Context.INPUT_METHOD_SERVICE) as android.view.inputmethod.InputMethodManager
                imm.hideSoftInputFromWindow(v.windowToken, 0)
                spinnerTip.requestFocus()
                true
            } else {
                false
            }
        }

        spinnerTip.setOnItemClickListener { parent, view, position, id ->
```

```
            calculateTip()
        }

        switchRoundUp.setOnCheckedChangeListener { buttonView, isChecked ->
            calculateTip()
        }
    }

    private fun calculateTip() {
        val amountInput = etBillAmount.text.toString()
        var amount = 0.0

        try {
            amount = amountInput.toDouble()
        } catch (e: Exception) {
            amount = 0.0
        }

        val tipInput = spinnerTip.text.toString()
        var tipPercent = 15.0

        if (tipInput == "15%") {
            tipPercent = 15.0
        } else if (tipInput == "18%") {
            tipPercent = 18.0
        } else if (tipInput == "20%") {
            tipPercent = 20.0
        }

        var tip = (tipPercent / 100) * amount

        if (switchRoundUp.isChecked == true) {
            tip = ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance().format(tip)
        tvTipAmount.text = "Tip Amount: " + formattedTip
    }
}
```

activity_main.xml

Tabel 2. Source Code activity_main XML Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<ScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="40dp"
        android:gravity="center_vertical">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginBottom="16dp"
            android:text="Calculate Tip"
            android:textSize="16sp"/>


<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.Material3.TextInputLayout.FilledBox"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Bill Amount"
            app:startIconDrawable="@drawable/money"
            android:layout_marginBottom="24dp">


<com.google.android.material.textfield.TextInputEditText
                android:id="@+id/etBillAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="numberDecimal"
                android:singleLine="true"
                android:imeOptions="actionNext"/>

</com.google.android.material.textfield.TextInputLayout>


<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.Material3.TextInputLayout.FilledBox.E
xposedDropdownMenu"
```

```
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Tip Percentage"
            app:startIconDrawable="@drawable/percent"
            android:layout_marginBottom="32dp">

            <AutoCompleteTextView
                android:id="@+id/spinnerTipPercentage"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="none" />

</com.google.android.material.textfield.TextInputLayout>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:layout_marginBottom="32dp">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Round up tip?"
                android:textSize="16sp" />


<com.google.android.material.materialswitch.MaterialSwitch
                android:id="@+id/switchRoundUp"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content" />
        </LinearLayout>

        <TextView
            android:id="@+id/tvTipAmount"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Tip Amount: $0.00"
            android:textSize="38sp"
            android:textStyle="bold" />

        <Space
            android:layout_width="match_parent"
            android:layout_height="110dp" />

```

```
    </LinearLayout>
</ScrollView>
```

MainActivity.kt - Compose

Tabel 3. Source Code MainActivity Compose Soal 1

```
package com.example.modul2compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.annotation.DrawableRes
import androidx.annotation.StringRes
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.wrapContentWidth
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Switch
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
```

```
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.modul2compose.ui.theme.TipTimeTheme
import java.text.NumberFormat

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        enableEdgeToEdge()
        super.onCreate(savedInstanceState)
        setContent {
            TipTimeTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                ) {
                    TipTimeLayout()
                }
            }
        }
    }
}

@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf("") }
    var tipInput by remember { mutableStateOf("15") }
    var roundUp by remember { mutableStateOf(false) }

    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tipPercent = tipInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount, tipPercent, roundUp)

    Column(
        modifier = Modifier
            .padding(40.dp)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = stringResource(R.string.calculate_tip),
            modifier = Modifier
                .padding(bottom = 16.dp, top = 40.dp)
                .align(alignment = Alignment.Start)
        )
```

```
        EditNumberField(
            label = R.string.bill_amount,
            leadingIcon = R.drawable.money,
            keyboardOptions = KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Next
            ),
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        TipDropdownField(
            label = R.string.how_was_the_service,
            leadingIcon = R.drawable.percent,
            value = tipInput,
            onValueChanged = { tipInput = it },
            modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()
        )
        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )
        Text(
            text = stringResource(R.string.tip_amount, tip),
            style = MaterialTheme.typography.displaySmall
        )
        Spacer(modifier = Modifier.height(150.dp))
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    keyboardOptions: KeyboardOptions,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,
```

```
        leadingIcon = { Icon(painter = painterResource(id = leadingIcon), null) },
        singleLine = true,
        modifier = modifier,
        onValueChange = onValueChanged,
        label = { Text(stringResource(label)) },
        keyboardOptions = keyboardOptions
    )
}

@Composable
fun TipDropdownField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    var expanded by remember { mutableStateOf(false) }
    val options = listOf("15", "18", "20")

    Box(modifier = modifier) {
        TextField(
            value = "$value%",
            onValueChange = {},
            readOnly = true,
            label = { Text(stringResource(label)) },
            leadingIcon = { Icon(painter = painterResource(id = leadingIcon), null) },
            trailingIcon = {
                Icon(
                    painter = painterResource(id = R.drawable.arrow_drop_down),
                    contentDescription = null
                )
            },
            modifier = Modifier.fillMaxWidth()
        )
        Box(
            modifier = Modifier
                .matchParentSize()
                .clickable { expanded = true }
        )
        DropdownMenu(
            expanded = expanded,
            onDismissRequest = { expanded = false },
            modifier = Modifier.fillMaxWidth(0.8f)
```

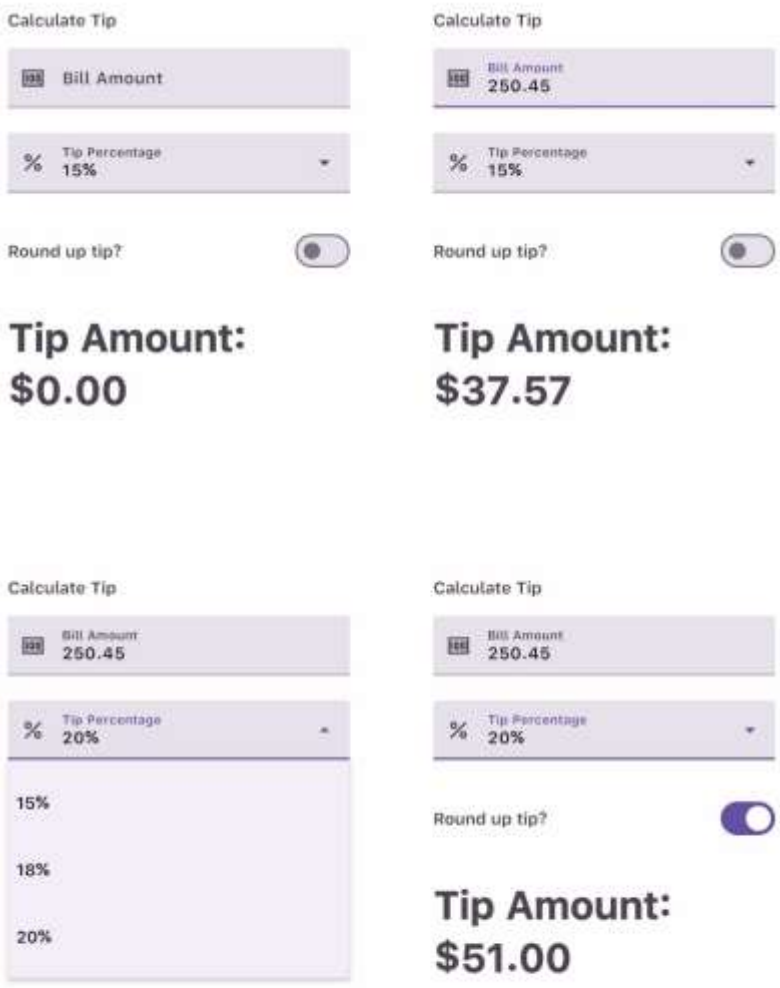
```
173         ) {
174             options.forEach { percentValue ->
175                 DropdownMenuItem(
176                     text = { Text("$percentValue%") },
177                     onClick = {
178                         onValueChanged(percentValue)
179                         expanded = false
180                     }
181                 )
182             }
183         }
184     }
185 }
186
187 @Composable
188 fun RoundTheTipRow(
189     roundUp: Boolean,
190     onRoundUpChanged: (Boolean) -> Unit,
191     modifier: Modifier = Modifier
192 ) {
193     Row(
194         modifier = modifier
195             .fillMaxWidth()
196             .size(48.dp),
197         verticalAlignment = Alignment.CenterVertically
198     ) {
199         Text(text = stringResource(R.string.round_up_tip))
200         Switch(
201             modifier = Modifier
202                 .fillMaxWidth()
203                 .wrapContentWidth(Alignment.End),
204             checked = roundUp,
205             onCheckedChange = onRoundUpChanged,
206         )
207     }
208 }
209
210 private fun calculateTip(
211     amount: Double,
212     tipPercent: Double = 15.0,
213     roundUp: Boolean
214 ): String {
215     var tip = tipPercent / 100 * amount
216     if (roundUp) {
217         tip = kotlin.math.ceil(tip)
218     }
219     return NumberFormat.getCurrencyInstance().format(tip)
```

```
}

@Preview(showBackground = true)
@Composable
fun TipTimeLayoutPreview() {
    TipTimeTheme {
        TipTimeLayout()
    }
}
```

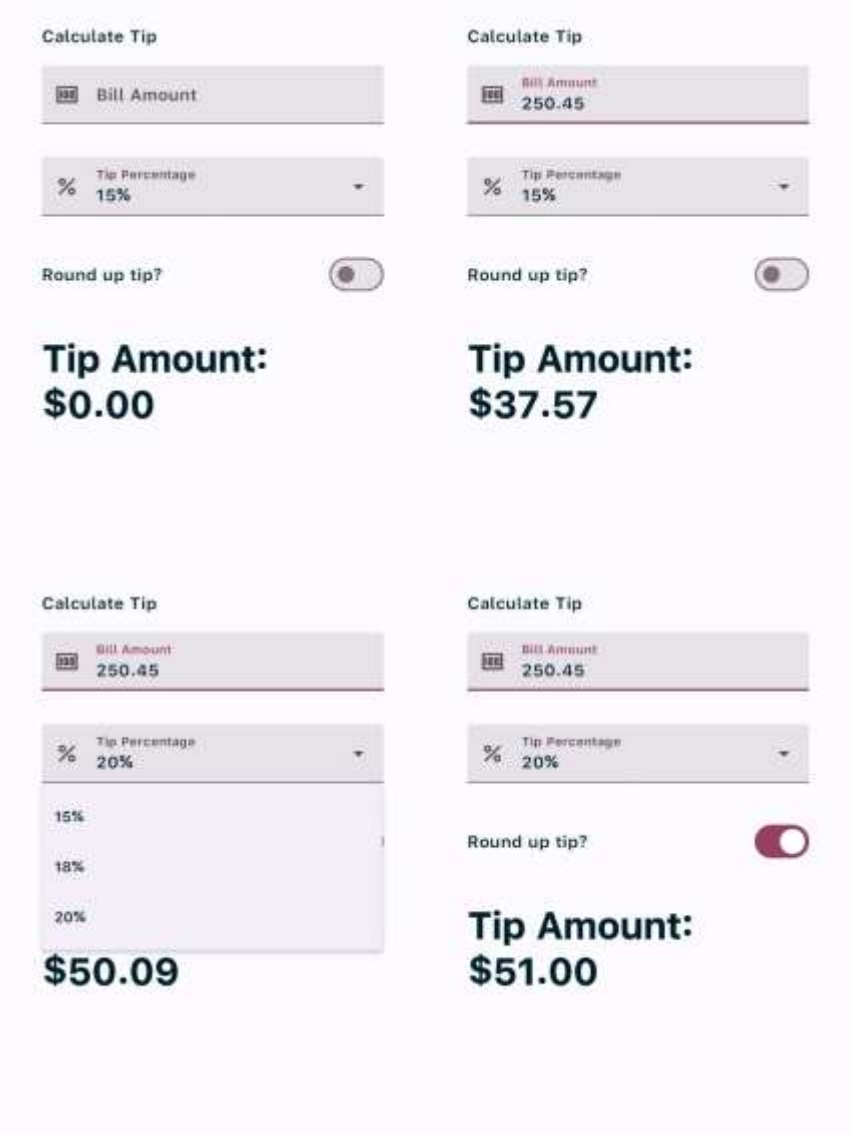
B. Output Program

XML



Gambar 1. Screenshot Hasil Jawaban Soal 1 XML

Compose



Gambar 2. Screenshot Hasil Jawaban Soal 1 Compose

C. Pembahasan

MainActivity.kt - XML

Kode ini merupakan program aplikasi Android penghitung uang tip yang dibangun menggunakan bahasa Kotlin dengan antarmuka berbasis XML. Program ini berfungsi untuk menghitung secara otomatis nominal tip yang harus dibayarkan berdasarkan input total tagihan dari pengguna, persentase tip yang dipilih, dan opsi tambahan untuk membulatkan hasil perhitungan ke atas.

Pada bagian ‘onCreate’, program terlebih dahulu menghubungkan komponen-komponen antarmuka dari file layout XML ke dalam variabel di kelas Kotlin menggunakan fungsi ‘findViewById’. Terdapat empat komponen utama yang diinisialisasi, yaitu ‘etBillAmount’ untuk kolom input tagihan, ‘spinnerTip’ untuk dropdown pilihan persentase, ‘switchRoundUp’ untuk sakelar pembulatan, dan ‘tvTipAmount’ untuk menampilkan hasil akhir. Setelah komponen terhubung, program menyiapkan daftar pilihan persentase tip (“15%”, “18%”, “20%”) ke dalam ‘spinnerTip’ menggunakan ‘ArrayAdapter’ dan mengatur nilai default tampilan awalnya ke “15%”. Lalu ada beberapa listener yang ditambahkan pada komponen-komponen tersebut untuk membuat aplikasi responsif secara real-time. Pada ‘etBillAmount’, dipasang ‘TextWatcher’ yang bertugas mengamati perubahan teks; fungsi ‘calculateTip’ akan langsung dipanggil pada blok ‘afterTextChanged’ setiap kali pengguna mengetikkan angka.

Selain itu, ‘setOnEditorActionListener’ digunakan untuk menangani aksi tombol “Next” pada keyboard, di mana sistem akan menyembunyikan keyboard dan memindahkan fokus ke dropdown ‘spinnerTip’. Fungsi ‘calculateTip’ juga akan otomatis terpicu ketika pengguna memilih opsi baru pada ‘spinnerTip’ lewat ‘setOnItemClickListener’, atau ketika pengguna menggeser ‘switchRoundUp’ lewat ‘setOnCheckedChangeListener’.

Fungsi ‘calculateTip’ merupakan inti logika dari program ini. Pertama, fungsi tersebut akan mengambil nilai string dari ‘etBillAmount’ dan mengubahnya menjadi tipe Double. Untuk mencegah aplikasi error (crash) jika input kosong atau formatnya salah, proses konversi ini dibungkus dalam blok ‘try-catch’ yang akan memberikan nilai default 0.0 jika terjadi kegagalan. Selanjutnya, program membaca teks persentase yang sedang aktif pada ‘spinnerTip’ dan menggunakan percabangan ‘if-else’ berantai untuk menentukan nilai variabel ‘tipPercent’ menjadi 15.0, 18.0, atau 20.0 sesuai pilihan.

Setelah nilai tagihan dan persentase didapatkan, program menghitung nominal tip dengan rumus persentase dibagi 100 lalu dikalikan dengan jumlah tagihan. Kemudian terdapat pengecekan kondisi ‘isChecked’ pada ‘switchRoundUp’, jika sakelar aktif (bernilai true), maka hasil hitungan tip akan dibulatkan ke bilangan bulat terdekat di atasnya menggunakan fungsi ‘ceil’. Terakhir, angka hasil perhitungan tersebut diformat menjadi bentuk standar mata uang menggunakan ‘NumberFormat.getCurrencyInstance()’ dan langsung ditampilkan ke layar dengan memperbarui teks pada komponen “tvTipAmount’.

activity_main.xml

File XML ini bertugas sebagai penyusun antarmuka (UI) untuk aplikasi penghitung tip. Sebagai fondasi paling luar, strukturnya dibungkus menggunakan ‘ScrollView’ dengan atribut ‘fillViewport=“true”‘. Tujuannya agar halaman tetap bisa digulir (scroll) seandainya ukuran layar perangkat (misalnya HP yang layarnya kecil) tidak cukup untuk menampilkan semua konten sekaligus. Di dalamnya, terdapat sebuah LinearLayout berorientasi vertikal yang menjadi wadah utama untuk menumpuk semua komponen antarmuka dari atas ke bawah secara berurutan. Layout ini diberi padding 40dp agar tampilannya lebih rapi, tidak menempel di tepi layar, dan diposisikan ke tengah (center_vertical).

Pada bagian input pengguna, komponen yang dipakai diambil dari library Material Design agar tampilannya lebih modern. Untuk memasukkan jumlah tagihan, digunakan perpaduan ‘TextInputLayout’ dan ‘TextInputEditText’. Bagian ini sengaja disetel dengan ‘inputType=“numberDecimal”‘ agar keyboard yang muncul di HP langsung berupa angka desimal. Atribut ‘imeOptions=“actionNext”‘ juga ditambahkan agar saat tombol next di keyboard ditekan, fokus akan otomatis pindah ke input selanjutnya. Untuk input persentase tip, komponen yang digunakan adalah ‘AutoCompleteTextView’ dengan ‘ExposedDropdownMenu’. Kombinasi ini secara visual bertindak seperti dropdown menu atau spinner, di mana pengguna tinggal memilih angka persentase tanpa harus mengetiknya secara manual.

Lalu terdapat opsi pembulatan tip yang dibungkus dengan ‘LinearLayout’ berorientasi horizontal. Di layout ini, terdapat teks “Round up tip?” dan sebuah ‘MaterialSwitch’. Teks tersebut diberi atribut layout_’weight=“1”‘ yang fungsinya untuk mendorong switch mentok ke ujung kanan layar, sehingga tampilannya sejajar di kedua sisi dan rapi. Sebagai penampil hasil akhir, lalu ada sebuah ‘TextView’ dengan ID ‘tvTipAmount’ yang ukurannya dibuat besar (38sp) dan tebal (bold).

MainActivity.kt - Compose

Kode ini merupakan program aplikasi Android penghitung uang tip yang dibuat menggunakan Jetpack Compose dengan bahasa Kotlin. Program ini berfungsi untuk

menghitung nominal tip yang harus dibayarkan berdasarkan total tagihan, persentase tip yang dipilih, serta opsi tambahan untuk membulatkan hasil akhir ke atas.

Pada fungsi utama ‘TipTimeLayout’, status dari input pengguna dikelola menggunakan remember dan ‘mutableStateOf’. Terdapat tiga state utama yang dipantau: ‘amountInput’ untuk menyimpan angka tagihan yang diketik pengguna, ‘tipInput’ dengan nilai awal “15” untuk menyimpan persentase tip, dan ‘roundUp’ bertipe boolean untuk menyimpan status pembulatan. Penggunaan state ini memastikan antarmuka selalu memperbarui tampilannya secara otomatis setiap kali ada perubahan data. Teks input dari pengguna kemudian dikonversi ke tipe data Double melalui ‘toDoubleOrNull’ agar bisa diproses ke dalam perhitungan matematika.

Secara visual, antarmukanya disusun vertikal menggunakan Column. Di dalamnya dipanggil fungsi ‘EditNumberField’ yang menampilkan kolom teks khusus (keyboard type number) untuk memasukkan jumlah tagihan. Untuk persentasenya, pengguna tidak perlu mengetik manual melainkan memilih lewat ‘TipDropdownField’. Komponen ini memanfaatkan Box, ‘TextField’ bersifat read-only, dan ‘DropdownMenu’ untuk memunculkan daftar pilihan statis yakni 15%, 18%, dan 20% ketika area tersebut ditekan.

Tepat di bawah dropdown, terdapat fungsi ‘RoundTheTipRow’ yang berisi komponen Row dengan teks dan sebuah Switch. Switch ini terhubung langsung dengan state ‘roundUp’, jika digeser menjadi aktif, maka statusnya berubah menjadi true. Setiap kali terjadi interaksi pada input tagihan, pilihan dropdown, maupun pergeseran switch, nilai-nilainya akan langsung dikirim ke fungsi ‘calculateTip’.

Fungsi ‘calculateTip’ bertugas memproses perhitungan dengan membagi persentase tip dengan 100 lalu mengalikannya dengan jumlah tagihan. Terdapat logika pengecekan kondisi roundup yang apabila bernilai true, hasil perhitungan akan dibulatkan ke bilangan bulat terdekat di atasnya menggunakan fungsi kotlin.math.ceil. Setelah proses hitung selesai, angka tersebut diformat ke dalam bentuk mata uang standar menggunakan ‘NumberFormat.getCurrencyInstance()’ dan langsung ditampilkan sebagai teks di layar.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Perbedaan XML dan Jetpack Compose

Perbedaan paling mendasar dari kedua implementasi ini dapat dilihat pada paradigma penulisannya. Pada XML, desain antarmuka yang berada di file .xml dan logika program yang ada di file Kotlin harus dipisahkan secara tegas. Dalam praktiknya, komponen-komponen antarmuka tersebut harus diperintah satu per satu secara manual. Misalnya, ID komponen dicari terlebih dahulu menggunakan 'findViewById', lalu listener dipasang, hingga akhirnya teks pada komponen tersebut diinstruksikan untuk berubah. Sebaliknya, pendekatan deklaratif yang digunakan oleh Jetpack Compose, yang mana tampilan dan logikanya digabungkan ke dalam satu file Kotlin. Bentuk antarmuka cukup dideskripsikan berdasarkan suatu data atau state. Ketika data tersebut mengalami perubahan misalnya melalui pelacakan oleh remember maka antarmuka akan digambar ulang (recompose) secara otomatis tanpa perlunya instruksi manual. Pendekatan ini jauh lebih ringkas sehingga penulisan kode repetitif (boilerplate) yang biasanya wajib ditulis pada XML dapat dipangkas secara drastis.

Kelebihan dan Kekurangan Implementasi XML

Dari segi kelebihan, pemisahan struktur (Separation of Concerns) yang sangat jelas antara kerangka visual dan logika sistem ditawarkan oleh implementasi XML. Navigasi kode dirasa lebih dimudahkan oleh sebagian orang, karena file yang harus dibuka untuk sekadar mengubah warna atau ukuran tombol sudah dapat diketahui secara pasti. Selain itu, karena XML sudah digunakan selama belasan tahun, dokumentasi, referensi, serta solusi perbaikan bug sangat mudah ditemukan di berbagai forum seperti StackOverflow. Namun, kekurangan yang cukup merepotkan juga dimiliki oleh XML. Aplikasi sangat rawan mengalami crash ('NullPointerException') jika ID komponen salah diketik atau lupa disamakan antara file XML dan Kotlin. Selain itu, banyak baris kode sering tersita hanya untuk mengurus hal-hal sederhana, seperti pembuatan dropdown pilihan persentase tip yang diwajibkan untuk diatur menggunakan array dan ArrayAdapter secara manual agar datanya bisa muncul di layar.

Kelebihan dan Kekurangan Implementasi Jetpack Compose

Di sisi lain, kepraktisan penulisan dijadikan keunggulan utama yang ditawarkan oleh Jetpack Compose, karena kerumitan 'findViewById' atau konfigurasi 'ViewBinding' sudah sepenuhnya ditinggalkan. Antarmuka murni ditulis menggunakan bahasa Kotlin, sehingga logika dasar seperti 'if-else' atau perulangan 'for' dapat langsung diselipkan ke dalam struktur tampilan. Pembuatan komponen yang dinamis atau animasi transisi juga jauh lebih mudah dieksekusi melalui implementasi ini. Namun, kekurangan terbesar Compose sering dirasakan pada kurva pembelajarannya yang lumayan curam bagi pemula atau mereka yang sudah terlalu terbiasa dengan XML. Pola pikir dituntut untuk dirombak sepenuhnya agar cara kerja State dan siklus hidup layar dapat dipahami dengan baik. Jika state salah ditulis, antarmuka dapat ter-render berulang kali secara tidak wajar sehingga memori perangkat akan sangat terbebani. Selain itu, karena teknologinya masih relatif baru, solusi untuk bug antarmuka yang spesifik terkadang belum banyak dibahas oleh komunitas pemrogram di internet.

ADDITIONAL

1. Apa saja fitur yang menjadi daya jual pada Kotlin?**
2. Berikan 1 test case terkait penggunaan fitur unggulan yang ada pada Kotlin

---

1. Fitur Unggulan Kotlin

Kotlin memiliki sejumlah fitur unggulan yang membedakannya dari bahasa pemrograman lain, khususnya Java.

1) **Null Safety**

Ini adalah salah satu fitur paling krusial di Kotlin. Menurut Talib dkk. [1], Kotlin memiliki nullable type system yang memungkinkan developer menulis kode yang lebih aman sehingga dapat mencegah terjadinya null pointer exceptions yang biasanya menjadi sumber utama bug di banyak program. Hal ini juga sejalan dengan yang dicatat oleh Hecht dan Bergel [2], yang menyebutkan bahwa referensi non-nullable secara default merupakan salah satu perbedaan mendasar Kotlin dibanding Java, di samping sintaksnya yang lebih ringkas.

2) **Concise Syntax (Sintaks Ringkas)**

Hecht dan Bergel [2] mencatat bahwa Kotlin memang sejak awal didesain dengan mempertimbangkan interoperabilitas terhadap Java, sehingga proses memanggil kode Java dari Kotlin maupun sebaliknya dapat dilakukan dengan mudah. Hal ini juga ditegaskan oleh Talib dkk. [1] yang menyatakan bahwa fitur ini memungkinkan developer untuk menggunakan Kotlin dan Java secara bersamaan di dalam satu proyek tanpa masalah berarti, sehingga memudahkan kolaborasi maupun migrasi bertahap dari codebase lama.

3) **Interoperability dengan Java**

Hecht dan Bergel [2] mencatat bahwa Kotlin memang sejak awal didesain dengan mempertimbangkan interoperabilitas terhadap Java, sehingga proses memanggil kode Java dari Kotlin maupun sebaliknya dapat dilakukan dengan mudah. Hal ini juga ditegaskan oleh Talib dkk. [1] yang menyatakan bahwa fitur ini memungkinkan developer untuk menggunakan Kotlin dan Java secara bersamaan di dalam satu proyek

tanpa masalah berarti, sehingga memudahkan kolaborasi maupun migrasi bertahap dari codebase lama.

4) **Data Classes**

Berdasarkan tabel fitur yang disusun oleh Talib dkk. [1], Kotlin mendukung data classes dengan catatan bahwa fitur ini membutuhkan jauh lebih sedikit baris kode dibandingkan bahasa lain seperti Java. Hecht dan Bergel [2] juga mengidentifikasi data classes sebagai salah satu pembeda teknis antara Kotlin dan Java yang cukup signifikan.

5) **Extension Functions**

Tabel fitur yang disusun oleh Talib dkk. [1], Kotlin secara bawaan mendukung *extension functions*. Fitur ini memungkinkan developer untuk menambahkan fungsi baru ke sebuah class yang sudah ada tanpa harus mengubah source code class tersebut maupun melakukan inheritance.

6) **Coroutines**

Talib dkk. [1] mencantumkan coroutines dalam daftar fitur yang didukung oleh Kotlin. Fitur ini memungkinkan penulisan kode asynchronous dengan cara yang lebih sederhana dan mudah dipahami dibandingkan pendekatan thread konvensional.

7) **Type Inference**

Talib dkk. [1] mencantumkan *type inference for variable and property types* sebagai salah satu fitur yang didukung Kotlin. Dengan fitur ini, developer tidak harus selalu mendeklarasikan tipe variabel secara manual karena compiler dapat menentukannya sendiri dari konteks kode, sehingga kode yang dihasilkan menjadi lebih bersih dan ringkas.

8) **Adopsi yang Terus Meningkat di Ekosistem Android**

Hecht dan Bergel [2] dalam studi mereka terhadap lebih dari 200.000 APK menemukan bahwa proporsi aplikasi yang menggunakan Kotlin tumbuh dari 0,24% pada tahun 2017 menjadi 17,00% pada tahun 2020. Tren ini menunjukkan bahwa Kotlin semakin banyak dipilih oleh developer Android, terutama pada aplikasi-aplikasi populer yang cenderung lebih cepat mengadopsi Kotlin dibanding aplikasi pada umumnya.

2. **Test Case: Penggunaan Fitur Null Safety**

Skenario: Sebuah aplikasi ingin menampilkan nama pengguna yang didapat dari database. Nama tersebut bisa saja bernilai null karena pengguna belum mengisi profil.

Kode tanpa penanganan null

```
String name = getNameFromDatabase();
System.out.println(name.length());
```

Kode dengan Null Safety di Kotlin

```
val nameFromDb: String? = getNameFromDatabase()

println(nameFromDb?.length)

val displayName: String = nameFromDb ?: "Pengguna Anonim"
println(displayName)
```

Pembahasan:

Pada contoh kode Java, variabel 'name' bertipe 'String' biasa yang secara diam-diam bisa saja berisi 'null'. Compiler Java tidak memberikan peringatan apapun soal ini, sehingga ketika 'getNameFromDatabase()' mengembalikan 'null' dan baris 'name.length()' dieksekusi, program langsung berhenti dengan 'NullPointerException' di runtime. Yang membuat jenis error ini cukup berbahaya adalah karena ia tidak muncul di tahap kompilasi, melainkan baru ketika program sudah berjalan dan dipakai pengguna.

Kotlin menangani masalah ini lewat sistem tipenya yang membedakan variabel yang boleh 'null' dan yang tidak. Seperti yang dipaparkan Talib dkk. [1], nullable type system pada Kotlin memungkinkan developer membedakan kedua jenis variabel tersebut, sehingga potensi null pointer exception bisa ditangani sebelum program dijalankan. Dalam kode Kotlin di atas, 'nameFromDb' dideklarasikan dengan tipe 'String?', di mana tanda tanya menandakan variabel ini boleh bernilai 'null'. Jika tanda tanya tidak dituliskan, compiler Kotlin akan langsung menolak apabila ada kemungkinan nilai 'null' masuk ke variabel tersebut.

Operator '?.' atau safe call operator pada baris 'nameFromDb?.length' bekerja dengan cara memeriksa dulu apakah variabel bernilai 'null' sebelum mengakses propertinya. Bila 'nameFromDb' memang 'null', ekspresi tersebut tidak akan crash, melainkan mengembalikan 'null' begitu saja sehingga program tetap bisa berjalan.

Operator ‘?:’ atau elvis operator pada baris berikutnya digunakan untuk menetapkan nilai pengganti jika variabel ternyata ‘null’. Apabila ‘nameFromDb’ tidak ‘null’, nilainya akan dipakai langsung. Namun jika ‘null’, maka string ‘"Pengguna Anonim"‘ yang akan digunakan. Variabel ‘displayName’ bertipe ‘String’ tanpa tanda tanya, yang berarti setelah baris ini sudah dipastikan nilainya tidak akan pernah ‘null’ dan aman untuk diproses selanjutnya.

Talib dkk. [1] menegaskan bahwa fitur null safety inilah yang menjadikan Kotlin lebih unggul dalam hal keamanan kode dibandingkan Java, karena potensi ‘NullPointerException’ ditangani secara sistematis dari awal penulisan kode, bukan hanya mengandalkan kehati-hatian developer.

DAFTAR PUSTAKA

[1] M. S. Talib, A. M. Al-Najar, A. H. Hassan and Z. S. Talib, "Kotlin Programming Language: A Comprehensive Overview of Definition, Applications, Advantages, and Limitations," *International Journal of Computer Applications Technology and Research,* vol. 13, no. 3, pp. 7-9, 2024.

[2] G. Hecht and A. Bergel, "Quantifying the adoption of Kotlin on Android stores: Insight from the bytecode," in *Proceedings of 8th IEEE/ACM International Conference on Mobile Software Engineering and Systems (MOBILESoft'21)*, 2021.

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat
https://github.com/Fiezz65/MODUL-MOBILE/tree/master/MODUL_2

Berikut adalah tautan untuk source code yang telah dibuat.

MainActivity.kt - XML:
https://github.com/Fiezz65/MODUL-MOBILE/blob/master/MODUL_2/XML/app/src/main/java/com/example/modul2xml/MainActivity.kt

activity_main.xml:
https://github.com/Fiezz65/MODUL-MOBILE/blob/master/MODUL_2/XML/app/src/main/res/layout/activity_main.xml

MainActivity.kt - Compose:
https://github.com/Fiezz65/MODUL-MOBILE/blob/master/MODUL_2/COMPOSE/app/src/main/java/com/example/modul2compose/MainActivity.kt